

Lista de Permisos AWS para Terraform y GitHub Actions (laudacanceleatiempo)

Para que la aplicación funcione y se despliegue en AWS mediante infraestructura gestionada por Terraform y pipelines de CI/CD (GitHub Actions), el usuario / rol de IAM de ejecución precisa los siguientes permisos (Actions), agrupados por recurso:

1. Redes y VPC (Amazon EC2 y VPC)

Terraform se encarga de crear la VPC, subredes, tablas de enrutamiento, Internet Gateways y NAT Gateways.

- `ec2:Describe*` (Leer el estado de la VPC)
- `ec2:CreateVpc` , `ec2:DeleteVpc` , `ec2:ModifyVpcAttribute`
- `ec2:CreateSubnet` , `ec2:DeleteSubnet` , `ec2:ModifySubnetAttribute`
- `ec2:CreateInternetGateway` , `ec2:AttachInternetGateway` , `ec2:DeleteInternetGateway`
- `ec2:AllocateAddress` , `ec2:ReleaseAddress`
- `ec2:CreateNatGateway` , `ec2:DeleteNatGateway`
- `ec2:CreateRouteTable` , `ec2:CreateRoute` , `ec2:AssociateRouteTable` , `ec2:DeleteRouteTable`
- `ec2:CreateSecurityGroup` , `ec2:AuthorizeSecurityGroupIngress` , `ec2:AuthorizeSecurityGroupEgress` , `ec2:DeleteSecurityGroup`
- `ec2:CreateTags` , `ec2:DeleteTags`

2. Base de Datos (Amazon RDS)

Para aprovisionar la instancia y los grupos de subred de base de datos.

- `rds:CreateDBInstance` , `rds:ModifyDBInstance` , `rds:DeleteDBInstance` , `rds:DescribeDBInstances`
- `rds:CreateDBSubnetGroup` , `rds:DeleteDBSubnetGroup` , `rds:DescribeDBSubnetGroups`
- `rds:ListTagsForResource` , `rds:AddTagsToResource` , `rds:RemoveTagsFromResource`

3. Permisos y Roles (AWS IAM)

Terraform necesita crear el rol de ejecución (`ecs-execution-role`) y de tarea (`ecs-task-role`) de tu servicio en ECS.

- `iam:CreateRole` , `iam:DeleteRole` , `iam:GetRole` , `iam:PassRole`
- `iam:PutRolePolicy` , `iam:DeleteRolePolicy` , `iam:AttachRolePolicy` , `iam:DetachRolePolicy`
- `iam:ListInstanceProfilesForRole` , `iam:ListAttachedRolePolicies` , `iam:ListRolePolicies`
- `iam:TagRole`

4. Registro de Contenedores (Amazon ECR)

Terraform crea el repositorio destino y GitHub Actions compila y sube el contenedor.

- `ecr:CreateRepository` , `ecr:DeleteRepository` , `ecr:DescribeRepositories`

- `ecr:GetAuthorizationToken` (Necesario para Docker login)
- `ecr:BatchCheckLayerAvailability` , `ecr:GetDownloadUrlForLayer`
- `ecr:InitiateLayerUpload` , `ecr:UploadLayerPart` , `ecr:CompleteLayerUpload`
- `ecr:PutImage` , `ecr:ListTagsForResource`

5. Orquestación de Contenedores (Amazon ECS)

Terraform y GitHub Actions crean y actualizan clúster Fargate, servicios y la definición de la tarea.

- `ecs>CreateCluster` , `ecs:DescribeClusters` , `ecs>DeleteCluster`
- `ecs:RegisterTaskDefinition` , `ecs:DescribeTaskDefinition` ,
`ecs:DeregisterTaskDefinition`
- `ecs>CreateService` , `ecs:UpdateService` , `ecs:DescribeServices` , `ecs>DeleteService`
- `ecs:TagResource` , `ecs:UntagResource`

6. Balanceador de Carga (Application Load Balancer - ELBv2)

Para enrutar el tráfico HTTP hacia los contenedores.

- `elasticloadbalancing>CreateLoadBalancer` ,
`elasticloadbalancing:DescribeLoadBalancers` ,
`elasticloadbalancing>DeleteLoadBalancer`
- `elasticloadbalancing>CreateTargetGroup` ,
`elasticloadbalancing:DescribeTargetGroups` ,
`elasticloadbalancing>DeleteTargetGroup` ,
`elasticloadbalancing:ModifyTargetGroupAttributes`
- `elasticloadbalancing>CreateListener` , `elasticloadbalancing:DescribeListeners` ,
`elasticloadbalancing>DeleteListener`
- `elasticloadbalancing:AddTags`

7. Variables y Secretos (AWS Systems Manager Parameter Store)

Manejo de secretos para la URI de la DB y Keycloak.

- `ssm:PutParameter` , `ssm:GetParameter` , `ssm:GetParameters` , `ssm>DeleteParameter`
- `ssm:AddTagsToResource` , `ssm:ListTagsForResource`

8. Logs e Infraestructura Auxiliar (AWS CloudWatch y STS)

Salida de logs y validación de cuenta de AWS en Terraform.

- `logs>CreateLogGroup` , `logs:DescribeLogGroups` , `logs:PutRetentionPolicy` ,
`logs>DeleteLogGroup` , `logs:ListTagsLogGroup`
- `sts:GetCallerIdentity` (Terraform require este permiso base)

Alternativa Rápida (Recomendada para Desarrollo/Pruebas)

Para entornos de desarrollo personal o cuando se está configurando la infraestructura por primera vez, es una práctica habitual dar **todos los permisos** temporales para evitar errores constantes de falta de

permisos y verificar que Terraform funcione correctamente.

Solo necesitas asignarle al usuario la política **AdministratorAccess** .

Pasos para asignar AdministratorAccess:

1. Inicia sesión en la **Consola de AWS** y ve al servicio **IAM** (Identity and Access Management).
2. En el panel izquierdo, haz clic en **Usuarios**.
3. Selecciona el usuario configurado para GitHub Actions/Terraform (aquel con el que creaste tus Access Keys).
4. Haz clic en el botón **Agregar permisos** (Add permissions) y selecciona **Agregar políticas directamente** (Attach policies directly).
5. En el buscador, busca y selecciona **AdministratorAccess** (con ícono naranja, gestionada por AWS).
6. Haz clic en **Siguiente** y luego en **Agregar permisos**.

⚠ **Nota de Seguridad:** La política **AdministratorAccess** permite efectuar cualquier acción (*) sobre cualquier recurso (*). Si este entorno de infraestructura llega a ser crítico (producción de un negocio real), se recomienda posteriormente restringir al usuario usando la lista detallada de los puntos 1 al 8. Para tu entorno y despliegue inicial, ¡es ideal para no quedarte atascado!